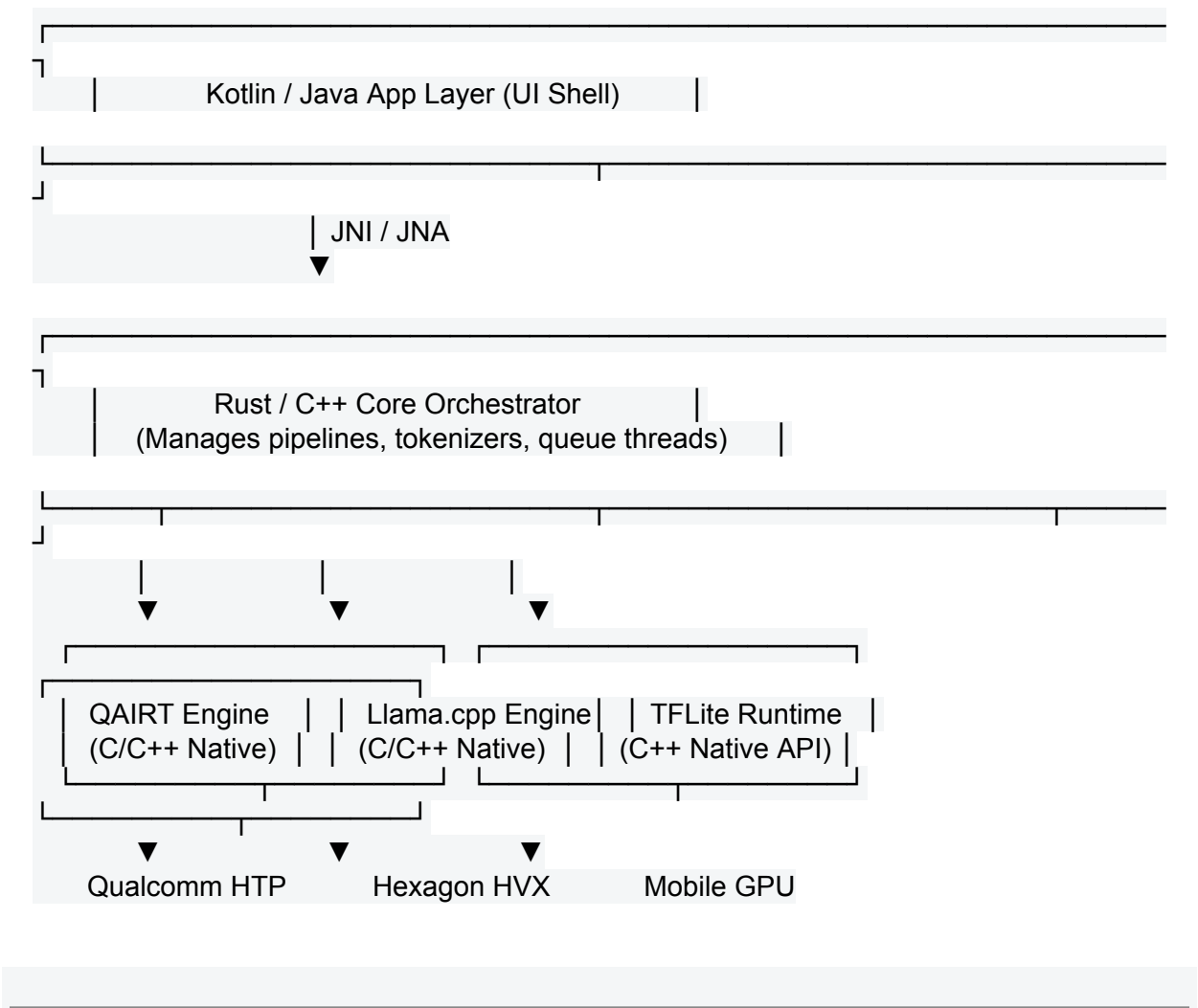


To support TFLite, GGUF (via QAI RT or llama.cpp), C++, and Rust in a single pluggable orchestrator, you must build a **native core architecture** using a C-compatible Foreign Function Interface (FFI).

If you try to orchestrate this directly in Kotlin/Java, your application will bog down in JNI Marshalling boilerplate, and adding new backends will break cross-platform portability.

The ideal industry pattern is to write the orchestrator engine core in **Rust** or **C++**, compile it as a unified shared library (.so), and let Kotlin/Java simply act as a shallow UI frontend shell.

1. The Multi-Language Orchestrator Architecture



2. The Core Native Interface (C-ABI)

Rust excels at bridging C++ libraries (like QAI RT and TFLite) to Java/Kotlin safely via the **Java Native Interface (JNI)** or **UniFFI**. [1]

Define your core orchestrator engine in Rust using standard C-ABI mappings so both C++ runtimes and Kotlin can interact with it without data copying overhead:

```
// src/lib.rs
use std::ffi::{CStr, CString};
use std::os::raw::{c_char, c_void};

#[repr(C)]
pub enum BackendType {
    QairtHtp = 0,
    LlamaCppGguf = 1,
    Tflite = 2,
}

// Global Orchestrator State managed safely inside Rust
pub struct Orchestrator {
    active_backend: BackendType,
    // Add runtime engine pointers here
}

#[no_mangle]
pub unsafe extern "C" fn orchestrator_create() -> *mut c_void {
    let orch = Box::new(Orchestrator { active_backend: BackendType::QairtHtp });
    Box::into_raw(orch) as *mut c_void
}

#[no_mangle]
pub unsafe extern "C" fn orchestrator_load_model(
    engine_ptr: *mut c_void,
    backend: i32,
    model_path: *const c_char
) -> i32 {
    let _orch = &mut *(engine_ptr as *mut Orchestrator);
    let path = CStr::from_ptr(model_path).to_str().unwrap_or("");

    match backend {
        0 => { /* Route path to Qualcomm QAI RT C++ loading binaries */ 0 },
        1 => { /* Route path to Llama.cpp raw GGUF context loads */ 0 },
        2 => { /* Route path to TFLite C++ Flatbuffer Interpreter */ 0 },
        _ => -1 // Unknown backend error
    }
}
```

```
}
```

3. The GitHub CI Multi-Language Matrix Toolchain

Because you are compiling C++, Rust, and Java/Kotlin simultaneously for Android, your GitHub CI configuration needs explicit toolchain staging targets.

Use this complete `.github/workflows/build-orchestrator.yml` workflow blueprint:

name: Build Cross-Language AI Orchestrator APK

on: [push, pull_request]

jobs:

build-native-core:

runs-on: ubuntu-latest

steps:

- **name:** Checkout Codebase
uses: actions/checkout@v4

- **name:** Setup Android NDK and SDK Environment
uses: android-actions/setup-android@v3

- **name:** Setup Rust Toolchain for Android Targets
run: |
rustup target add aarch64-linux-android
cargo install cargo-ndk

- **name:** Fetch Qualcomm QAI RT and TFLite Static Binaries
run: |
Pull down your native binary dependencies
aws s3 cp s3://my-deps/qairt-libs-v2.tar.gz . && tar -xzf qairt-libs-v2.tar.gz
aws s3 cp s3://my-deps/tflite-native-libs.tar.gz . && tar -xzf tflite-native-libs.tar.gz

- **name:** Compile C++/Rust Shared Library Core
run: |
Use cargo-ndk to compile the Rust core alongside custom C++ bindings
linked against QAI RT and TFLite native APIs
export QNN_SDK_ROOT=\$(pwd)/qairt
export TFLITE_SDK_ROOT=\$(pwd)/tflite

cargo ndk -t aarch64-linux-android -p 26 -- build --release

```
- name: Move Native Assets to Android App Project
run: |
  mkdir -p ./android/app/src/main/jniLibs/arm64-v8a
  cp ./target/aarch64-linux-android/release/libcore_orchestrator.so
  ./android/app/src/main/jniLibs/arm64-v8a/

- name: Compile Shallow Kotlin/Java APK Layer
run: |
  cd android
  chmod +x ./gradlew
  ./gradlew assembleRelease
```

4. Zero-Overhead Memory Mapping (Zero-Copy Data Transport)

When running heavy models like a **Qwen 3.5 9B** or complex **TFLite graphics models**, passing raw byte strings or memory segments back and forth across Kotlin $\rightleftharpoons$ Rust $\rightleftharpoons$ C++ will instantly run out of memory or thermal-throttle the SoC.

Enforce a **Memory File Descriptor (ashmem/memfd)** allocation loop inside the Orchestrator layout:

1. **Allocate Memory in Android:** Allocate a direct byte chunk in Kotlin using `ByteBuffer.allocateDirect()`. This creates memory space outside the standard garbage collected Java heap.
2. **Pass Address Pointer Natively:** Pass the direct memory pointer memory address directly to the Rust orchestrator via JNI.
3. **Hardware Zero-Copy:** Rust safely hands off this raw address straight into the QAI RT Tensor layer or TFLite's `Interpreter::tensor()` array.
4. **Result:** The NPU writes token outputs right into the space that Kotlin reads. Zero copy cycles occur during background inference.

Summary Architectural Matrix

Technology Component	Operational Target in Orchestrator	Primary Purpose
Kotlin / Java Layer	Android UI (Compose), State management.	Keeps code decoupled from core math modules.
Rust Native Core	JNI Binding Management, Safe Thread Pools.	Guarantees safety when hot-swapping memory

Technology Component	Operational Target in Orchestrator	Primary Purpose
		sectors.
C++ Core Extensions	Wraps raw QAI RT and TFLite static headers.	Direct, high-speed access to Qualcomm microcode drivers.
GitHub Actions Pipeline	cargo-ndk + cross-compiling linker scripts.	Combines three binary layouts into one artifact.

If you'd like, let me know:

- If you want to use **UniFFI** or raw **jni-sys bindings** in Rust to export the native bindings over to Kotlin automatically.
- Whether your TFLite models require a **GPU delegate execution path** alongside your GGUF NPU routines.

I can provide the boilerplate Cargo configs or CMake lists needed to link the C++ and Rust code cleanly inside your repo!

[1] <https://proandroiddev.com>